Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №1

по курсу «Моделирование»

«Построение аналитической статической модели»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Домрачева А.Б.

Москва 2026

1 Постановка задачи

Провести сгущение регулярной сетки размера $n \times n$: построить интерполяционную функцию по узлам исходной сетки, выполнить уточнение (добавление промежуточных точек), оценить среднюю абсолютную погрешность и визуализировать результат. Начальные условия: размер сетки n, интерполируемая функция $z = f(x, y)$, диапазоны значений x и y.

Исходные данные: $n = 10$ (по каждой оси n частей, сетка $(n+1) \times (n+1)$ узлов); диапазон по x: $[-3, 3]$, по y: $[-3, 3]$; интерполируемая функция $z = f(x, y) = \sin\left(\sqrt{x^2 + y^2}\right)$.

2 Построение математической модели

Исходные данные. Задана регулярная сетка узлов: по оси x точки $x_0 < x_1 < \ldots < x_n$, по оси y — $y_0 < y_1 < \ldots < y_n$. В каждом узле (x_i, y_j) задано значение z_{ij}. Область разбита на ячейки — прямоугольники $[x_i, x_{i+1}] \times [y_j, y_{j+1}]$.

Бикубический сплайн. Интерполяционная функция $S(x, y)$ строится как дважды непрерывно дифференцируемая функция ($S \in C^2$), обладающая следующими свойствами. На каждой ячейке $[x_i, x_{i+1}] \times [y_j, y_{j+1}]$ S является полиномом двух переменных степени не выше 3 по x и не выше 3 по y:

$$S(x, y) = \sum_{k=0}^{3} \sum_{l=0}^{3} a_{kl}^{(ij)} (x - x_i)^k (y - y_j)^l, \quad (x, y) \in [x_i, x_{i+1}] \times [y_j, y_{j+1}]. \tag{1}$$

Коэффициенты $a_{kl}^{(ij)}$ однозначно определяются из двух групп условий.

Условия интерполяции: в каждом узле сетки значение S совпадает с заданным,

$$S(x_i, y_j) = z_{ij}, \quad i = 0, \ldots, n, \; j = 0, \ldots, n. \tag{2}$$

Условия гладкого склеивания на границах соседних ячеек: на каждой вертикальной границе $x = x_{i+1}$ и горизонтальной границе $y = y_{j+1}$ совпадают значения самой функции и её частных производных до второго порядка включительно. Для границы по x между ячейками (i, j) и $(i+1, j)$ при $y \in [y_j, y_{j+1}]$

требуются равенства (индексы $\mp$ — пределы слева и справа по x):

$$
\begin{gathered}
S\big|_- = S\big|_+, \quad \left.\frac{\partial S}{\partial x}\right|_- = \left.\frac{\partial S}{\partial x}\right|_+, \quad \left.\frac{\partial S}{\partial y}\right|_- = \left.\frac{\partial S}{\partial y}\right|_+; \\
\left.\frac{\partial^2 S}{\partial x^2}\right|_- = \left.\frac{\partial^2 S}{\partial x^2}\right|_+, \quad \left.\frac{\partial^2 S}{\partial x \partial y}\right|_- = \left.\frac{\partial^2 S}{\partial x \partial y}\right|_+, \quad \left.\frac{\partial^2 S}{\partial y^2}\right|_- = \left.\frac{\partial^2 S}{\partial y^2}\right|_+.
\end{gathered} \tag{3}
$$

Аналогичные условия задаются на границах по y. Аналогичные условия задаются на границах по y. Число неизвестных коэффициентов $a_{kl}^{(ij)}$ (по 16 на ячейку) совпадает с числом линейных уравнений, получаемых из этих условий; система невырождена, поэтому S однозначно восстанавливается по сетке $\{z_{ij}\}$. В любой точке области значение $S(x, y)$ задаётся полиномиальной формулой (1) на той ячейке, которой принадлежит (x, y).

Сгущение сетки. Уточнённая сетка получается добавлением к исходным узлам середин отрезков: по x добавляются точки $\widetilde{x}_i = (x_i + x_{i+1})/2$, по y — $\widetilde{y}_j = (y_j + y_{j+1})/2$. В новых точках значение поля задаётся интерполяционной функцией: в точке $(\widetilde{x}_i, \widetilde{y}_j)$ полагается $z = S(x, y)$. В исходных узлах сохраняются заданные значения z_{ij}. В результате получается сетка с узлами по каждой оси в два раза плотнее исходной.

3 Разбиение на вычислительные задачи

3.1 Построение сетки и расчёт значений в узлах

Область $[-3, 3]$ по каждой оси разбивается на 10 равных частей, получаются координаты узлов по осям x и y. Все комбинации узлов дают регулярную сетку 10×10. В каждом узле вычисляется $z = f(x, y) = \sin(\sqrt{x^2 + y^2})$.

3.2 Бикубическая интерполяция и уточнение сетки

По узлам (x_i, y_j) и значениям z_{ij} строится бикубический сплайн. Координаты уточнённой сетки: исходные узлы и середины между соседними узлами. В узлах уточнённой сетки значения берутся из исходных данных (где узел совпадает с исходным) или со сплайна.

3.3 Оценка погрешности

Средняя абсолютная погрешность вычисляется по формуле

$$E_{\text{ср}} = \frac{1}{N}\sum_{k=1}^{N}\left|z_{\text{интерп}}(x_k, y_k) - f(x_k, y_k)\right|, \tag{4}$$

где N — число точек выбранного набора; (x_k, y_k) — координаты этих точек (исходные узлы и центры ячеек либо только центры ячеек); $z_{\text{интерп}}$ — значение интерполяционной функции в точке (x_k, y_k); f — точная функция. Считаются два варианта: с учётом исходных узлов и без них.

4 Анализ модели

Визуализация интерполяционной функции на уточнённой сетке в виде тепловой карты представлена на рисунке 1.

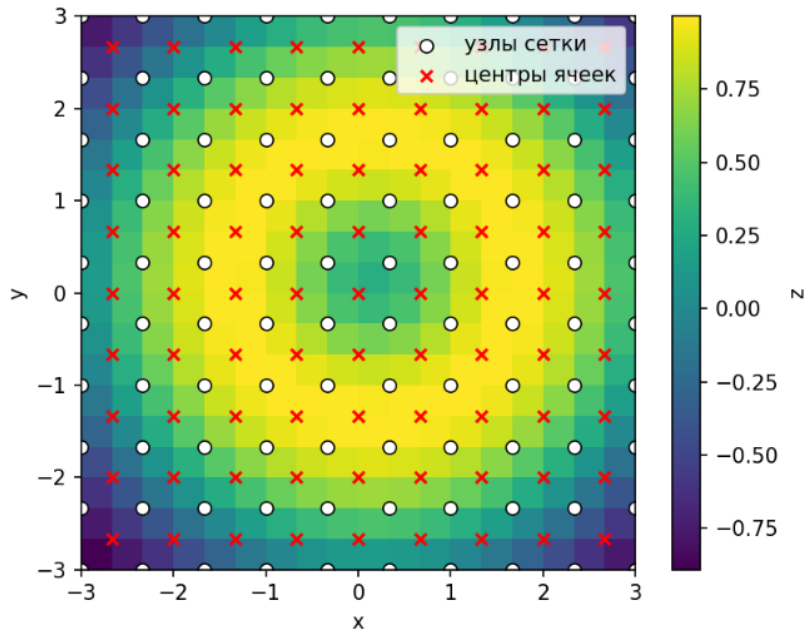

Рис. 1 — Узлы сетки и центры ячеек

Результаты расчёта средней абсолютной погрешности интерполяции: с учётом исходных узлов — 0,0036438606048676; без учёта исходных — 0,008141869202087699.

5 Вывод

В ходе лабораторной работы была спроектирована математическая модель для решения задачи сгущения сетки. Для решения задачи использовались бикубические сплайны. При анализе модели была проведена оценка работы модели. Модель отработала корректно, продемонстрировав высокую точность.

6 Практическая реализация

Листинг 1. Исходный код программы.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import RectBivariateSpline


def f(x, y):
    r = np.sqrt(x**2 + y**2)
    return np.sin(r)


def make_grid(nx, ny, x_min, x_max, y_min, y_max):
    x = np.linspace(x_min, x_max, nx + 1)
    y = np.linspace(y_min, y_max, ny + 1)
    return x, y


def get_refined_coords(x, y):
    nx, ny = len(x) - 1, len(y) - 1
    x_ref = np.empty(2 * nx + 1)
    y_ref = np.empty(2 * ny + 1)
    for i in range(nx):
        x_ref[2 * i] = x[i]
        x_ref[2 * i + 1] = 0.5 * (x[i] + x[i + 1])
    x_ref[2 * nx] = x[nx]
    for j in range(ny):
        y_ref[2 * j] = y[j]
        y_ref[2 * j + 1] = 0.5 * (y[j] + y[j + 1])
    y_ref[2 * ny] = y[ny]
    return x_ref, y_ref
```

```
def refine_grid_interp(x, y, z, interp):
    nx, ny = len(x) - 1, len(y) - 1
    centers_x = []
    centers_y = []
    z_centers = []
    for i in range(nx):
        for j in range(ny):
            xc = 0.5 * (x[i] + x[i + 1])
            yc = 0.5 * (y[j] + y[j + 1])
            zc = interp.ev(xc, yc)
            centers_x.append(xc)
            centers_y.append(yc)
            z_centers.append(zc)
    return np.array(centers_x), np.array(centers_y), np.array(
   z_centers), interp


def build_refined_grid(x, y, z, interp, levels=1):
    nx, ny = len(x) - 1, len(y) - 1
    x_ref, y_ref = get_refined_coords(x, y)
    nrx, nry = 2 * nx + 1, 2 * ny + 1
    z_ref = np.empty((nrx, nry))
    for i in range(nrx):
        for j in range(nry):
            if i % 2 == 0 and j % 2 == 0:
                z_ref[i, j] = z[i // 2, j // 2]
            else:
                z_ref[i, j] = interp.ev(x_ref[i], y_ref[j])
    if levels > 1:
        _, _, _, interp_next = refine_grid_interp(x_ref, y_ref,
   z_ref, RectBivariateSpline(x_ref, y_ref, z_ref, kx=3, ky=3, s=0)
   )
        return build_refined_grid(x_ref, y_ref, z_ref, interp_next,
    levels=levels - 1)
    return x_ref, y_ref, z_ref


def main():
    nx, ny = 9, 9
    x_min, x_max = -3.0, 3.0
    y_min, y_max = -3.0, 3.0
    x, y = make_grid(nx, ny, x_min, x_max, y_min, y_max)
    X, Y = np.meshgrid(x, y, indexing="ij")
    z = f(X, Y)
```

```
nodal_err = 1e-6
rng = np.random.default_rng(42)
z_noisy = z + rng.uniform(-nodal_err, nodal_err, z.shape)

X_nodes, Y_nodes = np.meshgrid(x, y, indexing="ij")
centers_x = np.array([0.5 * (x[i] + x[i + 1]) for i in range(nx
) for j in range(ny)])
centers_y = np.array([0.5 * (y[j] + y[j + 1]) for i in range(nx
) for j in range(ny)])
x_all = np.concatenate([X_nodes.ravel(), centers_x])
y_all = np.concatenate([Y_nodes.ravel(), centers_y])
z_true_nodes = f(x_all, y_all)
z_true_centers = f(centers_x, centers_y)

interp = RectBivariateSpline(x, y, z_noisy, kx=3, ky=3, s=0)
z_interp_all = interp.ev(x_all, y_all)
z_interp_centers = interp.ev(centers_x, centers_y)
mean_with = np.mean(np.abs(z_interp_all - z_true_nodes))
mean_without = np.mean(np.abs(z_interp_centers - z_true_centers
))
print("                                    :", mean_with)
print("                              :", mean_without)
centers_x, centers_y, z_centers, interp = refine_grid_interp(x,
y, z_noisy, interp)
x_ref, y_ref, z_ref = build_refined_grid(x, y, z_noisy, interp)

fig, ax = plt.subplots()
x_edges = np.concatenate([x_ref, [2 * x_ref[-1] - x_ref[-2]]])
y_edges = np.concatenate([y_ref, [2 * y_ref[-1] - y_ref[-2]]])
im = ax.pcolormesh(x_edges, y_edges, z_ref, shading="flat")
plt.colorbar(im, ax=ax, label="z")
X_nodes, Y_nodes = np.meshgrid(x, y, indexing="ij")
ax.scatter(X_nodes.ravel(), Y_nodes.ravel(), c="white", s=40,
edgecolors="black", linewidths=0.8, label="                    ",
 zorder=2)
ax.scatter(centers_x, centers_y, c="red", marker="x", s=30,
label="                          ", zorder=2)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_xlim(x_min, x_max)
ax.set_ylim(y_min, y_max)
ax.legend()
ax.set_aspect("equal")
fig.savefig("result.png", dpi=150)
plt.close(fig)
```

```
if __name__ == "__main__":
    main()
```